Experimentação — Sprint 4

Construção de um LLM do Zero · Pedro Devens Dellalibera e João Gabriel Trevisol

Testamos o modelo GPT completo (Entregável 3, `4,2.py`) variando cinco parâmetros da arquitetura, um de cada vez, mantendo os demais fixos na configuração base `GPT_CONFIG_124M`. Reaproveitamos a classe `GPTModel` e a função `contar_parametros` que já estavam prontas no `4,2.py`, e a função `generate_text_simple` do `4,3.py`.

Para cada configuração medimos duas coisas: o número total de parâmetros (soma ingênua e com weight tying) e o tempo médio de um forward pass.

Metodologia

- **Parâmetros:** são determinados inteiramente pela arquitetura, portanto não dependem de hardware nem de execução. Calculamos os valores pela fórmula de contagem de cada camada e conferimos que batem exatamente com o que o `4,2.py` imprimiu no baseline (163.009.536 / 124.412.160, ver print de 19/10).
- **Tempo de forward:** medido com `time.perf_counter()`, com uma passagem de aquecimento descartada e a média de 5 repetições sobre um lote de 2 sequências. Esse valor depende da máquina, então registramos abaixo o hardware utilizado.
- **Modo de avaliação:** todas as configurações rodam com `modelo.eval()` antes da medição, o que desativa o dropout e garante que a comparação isole o custo do forward.
- **Verificação da geração:** em cada configuração também chamamos `generate_text_simple` com 5 tokens, para confirmar que o mecanismo de geração funciona em todas elas.

Ambiente	Descrição
Execução	`CPU (o código não move tensores para GPU)`
Sistema	`Windows · Python 3.14`
Repetições por medição	`5 (mais 1 de aquecimento, descartada)`

Script utilizado

O laço abaixo está versionado no repositório em `src/Sprint 4/4,5_experimentos.py`. Ele não redefine nenhuma classe: carrega `4,2.py` e `4,3.py` pelo caminho com `importlib`, já que os nomes desses arquivos contêm vírgula e não podem ser importados com `import` comum.

```
import importlib.util, time
from pathlib import Path
import torch, tiktoken


def carregar(arquivo, nome):            # "4,2.py" tem vírgula no nome e
    caminho = Path(__file__).resolve().parent / arquivo   # não pode ser
    spec = importlib.util.spec_from_file_location(nome, caminho)  # importado
    modulo = importlib.util.module_from_spec(spec)      # com import comum
```

```
    spec.loader.exec_module(modulo)
    return modulo

gpt = carregar("4,2.py", "sprint4_modelo")
ger = carregar("4,3.py", "sprint4_geracao")

tokenizer = tiktoken.get_encoding("gpt2")
ids_texto = torch.tensor(tokenizer.encode("Era uma vez"))
lote = torch.stack([ids_texto, ids_texto])
REPETICOES = 5

overrides = {
    "1. Baseline":        {},
    "2. Modelo mini":     {"n_layers": 4},
    "3. Menos cabecas":   {"n_heads": 4},
    "4. Mais dropout":    {"drop_rate": 0.3},
    "5. Contexto menor":  {"context_length": 256},
}

for nome, override in overrides.items():
    cfg = {**gpt.GPT_CONFIG_124M, **override}
    torch.manual_seed(123)
    modelo = gpt.GPTModel(cfg)
    modelo.eval()                 # desliga o dropout: medimos só o forward

    total = gpt.contar_parametros(modelo)
    tying = total - sum(p.numel() for p in modelo.out_head.parameters())

    with torch.no_grad():
        modelo(lote)              # aquecimento, fora da medição
        inicio = time.perf_counter()
        for _ in range(REPETICOES):
            modelo(lote)
        tempo = (time.perf_counter() - inicio) / REPETICOES

        ger.generate_text_simple(  # confirma que a geração roda nesta config
            model=modelo, idx=ids_texto.unsqueeze(0),
            max_new_tokens=5, context_size=cfg["context_length"])

    print(f"{nome:<20} {total:>12,} {tying:>12,} {tempo:>9.4f}")
```

Experimento 1 — Baseline

- Configuração original do `GPT_CONFIG_124M`, usada como referência para os outros quatro.

O que testar: Confirmar que o modelo completo bate com os ~124M de parâmetros do nome “GPT-2 124M” quando consideramos weight tying.

Como fizemos: Instanciamos `GPTModel(GPT_CONFIG_124M)` sem nenhum override e chamamos `contar_parametros` duas vezes: uma somando tudo, outra descontando o `out_head` (que no GPT-2 real é a mesma matriz do `tok_emb`).

Métrica	Valor
Configuração	`GPT_CONFIG_124M original`
Parâmetros (soma ingênua)	`163.009.536`

Parâmetros (weight tying)	`124.412.160`
Tempo médio de 1 forward	`0,0417 s  (100% - referência)`

A soma ingênua conta o `out_head` (`Linear(768, 50257)`, sem bias) como uma matriz própria de 38.597.376 parâmetros, a mais cara isolada do modelo. Com weight tying — reaproveitando a matriz do `tok_emb` como `out_head`, como faz o GPT-2 original — o total cai para os 124.412.160 que dão nome ao modelo.

O tempo medido de 0,0417 s serve de referência para os outros quatro experimentos. Todas as comparações de tempo abaixo são expressas como porcentagem desse valor.

Experimento 2 — Modelo "mini" (n_layers: 12 → 4)

- Número de Transformer Blocks empilhados.

O que testar: O quanto reduzir de 12 para 4 blocos economiza em parâmetros, e se a economia é proporcional à razão 4/12.

Como fizemos: Trocamos apenas `n_layers` no dicionário de configuração; `tok_emb`, `pos_emb` e `out_head` ficam exatamente iguais ao baseline, mudando só o número de `TransformerBlock` empilhados em `trf_blocks`.

Métrica	Valor
Configuração alterada	`n_layers: 12 → 4`
Parâmetros (soma ingênua)	`106.324.992`
Parâmetros (weight tying)	`67.727.616`
Tempo médio de 1 forward	`0,0214 s  (51% do baseline)`

Cada Transformer Block tem exatamente 7.085.568 parâmetros (atenção + FeedForward + 2 LayerNorm), número que não muda com `n_layers`. A diferença de 56.684.544 parâmetros entre baseline e mini é exatamente 8 blocos a menos (8 × 7.085.568).

Mas a razão final não é 4/12: `tok_emb` (38.597.376) e `out_head` (38.597.376) são fixos e pesam muito no total, então o modelo mini tem 65% dos parâmetros do baseline, e não 33%. Isso mostra que os embeddings e a cabeça de saída dominam o orçamento de parâmetros quando o vocabulário é grande e o modelo é raso.

O tempo de forward caiu para 51% do baseline — e, pelo mesmo motivo, também não caiu para 33%. Contando as multiplicações-acumulações do forward para este lote (2 × 6 = 12 posições): cada bloco custa 7.077.888 operações por posição, e o `out_head` custa 12 × 768 × 50.257 = 463.168.512 operações fixas. O baseline soma ≈ 1,48 bilhão e o mini ≈ 803 milhões, o que prevê 54% — bem perto dos 51% medidos. É o mesmo efeito visto nos parâmetros: o `out_head` é um custo fixo que não desaparece ao remover blocos.

Experimento 3 — Menos cabeças (n_heads: 12 → 4)

- Número de cabeças de atenção dentro de cada `MultiHeadAttention`.

O que testar: Se reduzir `num_heads` muda o número de parâmetros do modelo, já que no Experimento 1 da Sprint 3 vimos que `head_dim = d_out / num_heads` apenas reparticiona o mesmo espaço.

Como fizemos: Trocamos `n_heads` de 12 para 4 (768 continua divisível por 4, então não dispara o `AssertionError` que vimos na Sprint 3).

Métrica	Valor
Configuração alterada	`n_heads: 12 → 4`
Parâmetros (soma ingênua)	`163.009.536`
Parâmetros (weight tying)	`124.412.160`
Tempo médio de 1 forward	`0,0499 s  (120% do baseline)`

Total idêntico ao baseline — confirma, na arquitetura completa, o que a Sprint 3 já tinha mostrado isoladamente na `MultiHeadAttention`: `W_query`, `W_key`, `W_value` e `out_proj` têm shape fixo por `emb_dim` (768 × 768), independentemente de quantas cabeças dividem esse espaço. `num_heads` muda `head_dim` (de 64 para 192) e a forma como o tensor é reorganizado no `forward`, não quantos pesos existem.

O tempo medido ficou em 120% do baseline, mas isso não indica que menos cabeças deixem o modelo mais lento: como o número de pesos e de operações é idêntico ao do baseline, esta configuração funciona na prática como um controle da medição. A diferença de 20% é, portanto, a margem de erro do método — ponto retomado na observação final.

Experimento 4 — Mais dropout (drop_rate: 0.1 → 0.3)

- Taxa de dropout usada na atenção, na FeedForward e na soma residual.

O que testar: Se o dropout aparece na contagem de parâmetros, e o que essa mudança afeta de fato num modelo ainda sem treino.

Como fizemos: Trocamos `drop_rate` para 0.3 e rodamos o mesmo forward pass. Como chamamos `modelo.eval()` antes de medir, o dropout fica desativado em todas as cinco configurações — garantindo que o tempo medido compare apenas o custo do forward, e não a aplicação da máscara aleatória.

Métrica	Valor
Configuração alterada	`drop_rate: 0.1 → 0.3`
Parâmetros (soma ingênua)	`163.009.536`

Parâmetros (weight tying)	`124.412.160`
Tempo médio de 1 forward	`0,0474 s  (114% do baseline)`

`nn.Dropout` não tem peso aprendível — é apenas uma máscara aleatória aplicada durante o treino, então não muda nada na contagem de parâmetros nem no forward em modo `eval()`. A diferença entre 0.1 e 0.3 só vai aparecer quando o modelo estiver em modo `train()` durante o pré-treinamento (fora do escopo desta sprint, assim como já tinha acontecido com `dropout=0.0` nos experimentos da Sprint 3).

O tempo medido ficou em 114% do baseline. Como o dropout está desativado por `eval()` e nenhum peso ou operação muda, esta configuração também funciona como controle: a diferença observada é ruído de medição, não efeito do `drop_rate`.

Experimento 5 — Contexto menor (context_length: 1024 → 256)

- Tamanho máximo de sequência que o modelo aceita (afeta `pos_emb` e a máscara causal).

O que testar: O quanto `context_length` pesa na contagem de parâmetros, comparado com a redução de `n_layers` do Experimento 2.

Como fizemos: Trocamos `context_length` para 256; isso muda o tamanho da camada `pos_emb` (`Embedding(context_length, emb_dim)`) e o tamanho do buffer de máscara causal em cada `MultiHeadAttention` — que, por ser registrado com `register_buffer`, não conta como parâmetro treinável.

Métrica	Valor
Configuração alterada	`context_length: 1024 → 256`
Parâmetros (soma ingênua)	`162.419.712`
Parâmetros (weight tying)	`123.822.336`
Tempo médio de 1 forward	`0,0315 s  (76% do baseline)`

A queda é de apenas 589.824 parâmetros (163.009.536 − 162.419.712), exatamente igual a 768 × (1024 − 256), a diferença no tamanho da `pos_emb`. É uma fração mínima do total (0,36%), bem menor que a queda do Experimento 2. Isso faz sentido: `context_length` só define o tamanho da tabela de posições, não mexe em nenhum peso dos Transformer Blocks.

Na prática, reduzir o `context_length` limita quantos tokens o modelo processa de uma vez e reduz o custo quadrático da atenção, como vimos na Sprint 3 — mas quase não libera memória de parâmetros.

O tempo medido ficou em 76% do baseline, mas esse ganho aparente não pode ser atribuído ao contexto menor. O lote de teste tem apenas 6 tokens, e a máscara é recortada com `[:num_tokens, :num_tokens]` dentro do `forward` — ou seja, a matriz de atenção é 6 × 6 nas duas configurações, independentemente de `context_length` valer 1024 ou 256. O custo quadrático depende do número

de tokens processados, não do limite declarado na configuração. A redução de 16× no custo da atenção só apareceria se as sequências realmente tivessem 1024 e 256 tokens. Como a variação de 24% está dentro da margem observada nos controles (Experimentos 3 e 4), o mais correto é tratá-la como ruído.

Resumo comparativo

Configuração	Parâmetros · weight tying · tempo
1. Baseline	`163.009.536 · 124.412.160 · 0,0417 s  (100%)`
2. Modelo mini	`106.324.992 ·  67.727.616 · 0,0214 s  ( 51%)`
3. Menos cabeças	`163.009.536 · 124.412.160 · 0,0499 s  (120%)`
4. Mais dropout	`163.009.536 · 124.412.160 · 0,0474 s  (114%)`
5. Contexto menor	`162.419.712 · 123.822.336 · 0,0315 s  ( 76%)`

Sobre a margem de erro das medições

Os Experimentos 3 e 4 acabaram funcionando como controles não planejados. Em ambos, o número de pesos e a quantidade de operações do forward são idênticos aos do baseline: mudar `n_heads` apenas reparticiona matrizes de mesmo tamanho, e o `drop_rate` não tem efeito com o modelo em `eval()`. Logo, os dois deveriam ter dado praticamente o mesmo tempo do baseline.

Como deram 120% e 114%, essa dispersão é a margem de erro do método — cerca de ±20% com 5 repetições num desktop comum, onde o sistema operacional, o escalonamento de frequência da CPU e outros processos influenciam a medição.

Isso muda a leitura dos resultados: a única diferença de tempo que está claramente acima do ruído é a do Experimento 2 (51% do baseline, quase metade). A variação do Experimento 5 (76%) é da mesma ordem de grandeza do ruído dos controles e, por isso, não sustenta nenhuma conclusão. Para medir efeitos dessa magnitude com confiança, seria preciso aumentar bastante o número de repetições e relatar a dispersão, e não só a média.

Observação final

Em todas as cinco configurações a geração com `generate_text_simple` rodou sem erro, confirmando que o mecanismo funciona independentemente dos parâmetros escolhidos. O texto em si, porém, continua sendo ruído, porque nenhum dos cinco modelos foi treinado — comparar o texto gerado entre configurações não é informativo nesta etapa. O que esta experimentação mede são a contagem de parâmetros e o tempo de forward.

Vale separar também os dois eixos de custo que aparecem aqui: a contagem de parâmetros mede quanta memória o modelo ocupa armazenado, enquanto o tempo de forward reflete o custo de processamento por passo. São grandezas diferentes, e o Experimento 5 é o exemplo mais claro dessa distinção — ele quase não muda a primeira, mas afeta bastante a segunda.